

ANSIBLE VARIABLES

- Ansible supports variables that can be used to store values that can then be reused throughout files in an Ansible project.
- This can simplify the creation and maintenance of a project and reduce the number of errors.
- Variables provide a convenient way to manage dynamic values for a given environment in your Ansible project. Examples of values that variables might contain include:
 - Users to create
 - Packages to install
 - Services to restart
 - Files to remove
 - Archives to retrieve from the internet

NAMING VARIABLES

- Variable names must start with a letter, and they can only contain letters, numbers, and underscores.

Ansible Variable Names — Valid vs Invalid

Invalid Variable Name	Valid Variable Name
web server	web_server
remote.file	remote_file
1st file	file_1
1st file	file1
remoteserver\$1	remote_server_1
remoteserver\$1	remote_server1

DEFINING VARIABLES

- Variables can be defined in a variety of places in an Ansible project.
- However, this can be simplified to three basic scope levels:
 - Global scope: Variables set from the command line or Ansible configuration
 - Play scope: Variables set in the play and related structures
 - Host scope: Variables set on host groups and individual hosts by the inventory, fact gathering, or registered tasks
- If the same variable name is defined at more than one level, the level with the highest precedence wins.

- **A narrow scope takes precedence over a wider scope: variables defined by the inventory are overridden by variables defined by the playbook, which are overridden by variables defined on the command line.**

VARIABLES IN PLAYBOOKS

- Variables play an important role in Ansible Playbooks because they ease the management of variable data in a playbook.
- Defining Variables in Playbooks When writing playbooks, you can define your own variables and then invoke those values in a task.
- For example, a variable named `web_package` can be defined with a value of `httpd`. A task can then call the variable using the `yum` module to install the `httpd` package.
- Playbook variables can be defined in multiple ways.
 - 1:** One common method is to place a variable in a `vars` block at the beginning of a playbook.

Example-1: Create 5 Users , define user names and password as Variables in PlayBook

``yaml

- name: Create users with same initial password

hosts: all

become: yes

vars:

users:

- user1

- user2

- user3

- user4

- user5

default_password: "ChangeMe123"

tasks:

- name: Create users with default password

user:

```

    name: "{{ item }}"

    password: "{{ default_password | password_hash('sha512') }}"

    state: present

    update_password: always

    password_expire: true

    loop: "{{ users }}"

    ...

```

- You can call variables by using `loop: "{ variable_name }"`

OR

- You can use `with_items: "{ variable_name }"`

2: It is also possible to define playbook variables in external files. In this case, instead of using a `vars` block in the playbook, the `vars_files` directive may be used, followed by a list of names for external variable files relative to the location of the playbook.

Example-2: Install Packages, Use Variables from external file.

install-packages/

└─ **vars.yml**

└─ **install.yml**

1: Define variables in external file.

vi vars.yml

packages:

- httpd
- vim
- git
- tree
- wget

2:Call Variables in PlayBook.

```
``yaml
```

```
- name: Install packages using external variable file
```

```
  hosts: all
```

```
  become: yes
```

```
  vars_files:
```

```
    - vars.yml
```

```
  tasks:
```

```
    - name: Install required packages
```

```
      yum:
```

```
        name: "{{ item }}"
```

```
        state: present
```

```
        with_items: "{{ packages }}"
```

```
....
```

HOST VARIABLES AND GROUP VARIABLES

- Inventory variables that apply directly to hosts fall into two broad categories: host variables apply to a specific host, and group variables apply to all hosts in a host group or in a group of host groups.
- **Host variables** take precedence over **group variables**, but variables defined by a playbook take precedence over both.
- One way to define host variables and group variables is to do it directly in the inventory file.
- This is an older approach and not preferred, but you may still encounter it.
 - Defining the `ansible_user` host variable for `demo.example.com`:

```
[servers]
demo.example.com ansible_user=joe
```

- Defining the user group variable for the servers host group.

```
[servers]

Dc1.example.com

Dc2.example.com

[servers:vars]

firewall_service: http

document_root: /var/www/html
```

Using Directories to Populate Host and Group Variables

- The preferred approach to defining variables for hosts and host groups is to create two directories, `group_vars` and `host_vars`, in the same working directory as the inventory file or directory.
- These directories contain files defining group variables and host variables, respectively.
- **IMPORTANT**: The recommended practice is to define inventory variables using `host_vars` and `group_vars` directories, and not to define them directly in the inventory files. To define group variables for the `servers group`, you would create a YAML file named `group_vars/servers`, and then the contents of that file would set variables to values using the same syntax as in a playbook.

User: tester

- Likewise, to define host variables for a particular host, create a file with a name matching the host in the `host_vars` directory to contain the host variables.
- The following examples illustrate this approach in more detail. Consider a scenario where there are two data centers to manage and the data center hosts are defined in the `~/project/ inventory` file:

```
[admin@station project]$ cat ~/project/inventory
[datacenter1]
demo1.example.com
demo2.example.com
[datacenter2]
demo3.example.com
demo4.example.com
[datacenters:children]
datacenter1 datacenter2
```

- If you need to define a general value for all servers in both data centers, set a group variable for the datacenters host group:

```
[admin@station project]$ cat ~/project/group_vars/datacenters  
package: httpd
```

OVERRIDING VARIABLES FROM THE COMMAND LINE

- Inventory variables are overridden by variables set in a playbook, but both kinds of variables may be overridden through arguments passed to the `ansible` or `ansible-playbook` commands on the command line.
- Variables set on the command line are called **extra variables**. Extra variables can be useful when you need to override the defined value for a variable for a one-off run of a playbook. For example:

```
[user@demo ~]$ ansible-playbook main.yml -e "package=apache"
```

USING ARRAYS AS VARIABLES

- Instead of assigning configuration data that relates to the same element (a list of packages, a list of services, a list of users, and so on), to multiple variables, administrators can use arrays. One consequence of this is that an array can be browsed.

1. Original Variables (Before Using Arrays)

Instead of defining many separate variables like:

```
``yaml  
user1_first_name: Bob  
user1_last_name: Jones  
user1_home_dir: /users/bjones  
  
user2_first_name: Anne  
user2_last_name: Cook  
user2_home_dir: /users/acook  
``
```

2. Rewriting as an Array (Dictionary)

A cleaner way is to define a ****dictionary of users****:

```
``yaml
users:
  bjones:
    first_name: Bob
    last_name: Jones
    home_dir: /users/bjones

  acook:
    first_name: Anne
    last_name: Cook
    home_dir: /users/acook
...

```

Example:

- name: Demonstrate dictionary-style user variables

hosts: servers

become: no

tasks:

- name: Show Bob's first name

debug:

msg: "Bob's first name is {{ users.bjones.first_name }}"

- name: Show Anne's home directory

debug:

msg: "Anne's home directory is {{ users['acook']['home_dir'] }}"

! ****Important Note About Dot Notation in Ansible****

When accessing dictionary variables, Ansible allows two syntaxes:

Dot notation

```
``yaml
users.bjones.first_name
...

```

Bracket notation

```
``yaml
users['bjones']['first_name']
...

```

Both work — ****but dot notation can sometimes cause problems****.

Why Dot Notation Can Be Risky

Dot notation may break if your dictionary keys have the same names as ****Python methods or attributes****, such as:

```
- `copy`
- `add`
- `discard`
- `keys`
``yaml
mydata.copy
...

```

Ansible might interpret this as the ****Python `.copy()` method****, not your variable key.

This can lead to confusing errors.

✓ Recommended Best Practice (Red Hat Guidance)

To avoid conflicts and make troubleshooting easier:

Use **one syntax consistently** across your entire project

Prefer **bracket notation** when keys might conflict with Python names

Example

```
``yaml
users['bjones']['first_name']
users['acook']['home_dir']
...

```

Bracket notation **never conflicts** with Python methods, so it is the safest choice in complex playbooks.

